

Git & GitHub for DevOps Complete Curriculum

Approach 1: Linear Lecture + Lab (5 Sessions)

Beginner-friendly | 1 Hour per Session | Theory First, Then Practice

Instructor Guide

Philosophy

This curriculum uses a **traditional classroom structure**: present theory first, then immediately apply it. Each session is modular — if a student misses Session 2, they can still follow Session 3 because concepts are reviewed.

Magic Rule: Every term is introduced with an analogy before the technical definition.

Pacing:

- 20 min: Lecture (slides + analogies)
- 25 min: Guided lab (instructor demos, students follow)
- 10 min: Independent exercise (students work alone)
- 5 min: Review + Q&A

Student prerequisites: None.

Required environment: Same as all approaches — see Environment Setup below.

Before You Start

Students must complete the Environment Setup section before Session 1. Reserve 30 minutes for this, or assign as pre-class homework.

Environment Setup Guide

Same setup for all approaches. Do this before Day 1.

Step 1: Check Your Operating System

If you see...	You have
<code>C:\</code> drive or Windows logo	Windows
Finder or <code>~/</code>	Mac
<code>apt-get</code> or <code>dnf</code>	Linux

Step 2: Install Git

Environment: Windows

1. Visit `https://git-scm.com/download/win`
2. Download and run the installer
3. Accept all defaults (click Next repeatedly)
4. Open **Git Bash** from the Start menu
5. Verify:

```
$ git --version
```

Expected Output:

```
git version 2.43.0.windows.1
```

(Version number may differ — that's okay)

Environment: Mac

1. Open **Terminal** (Cmd+Space, type "Terminal")
2. Run:

```
$ git --version
```

Expected Output:

```
git version 2.39.3
```

If prompted to install Developer Tools, click **Install**.

Environment: Linux (Ubuntu/Debian)

```
$ sudo apt update && sudo apt install git -y  
$ git --version
```

Expected Output:

```
git version 2.34.1
```

Environment: Linux (Fedora/CentOS/RHEL)

```
$ sudo dnf install git -y  
$ git --version
```

Step 3: Configure Git Identity

Environment: Any terminal (Git Bash / Terminal)

```
$ git config --global user.name "Your Full Name"  
$ git config --global user.email "youremail@example.com"
```

Analogy: Git needs to know who you are, like a library card needs your name.

Verify:

```
$ git config --global user.name  
$ git config --global user.email
```

Step 4: Create GitHub Account

Environment: Web Browser

1. Go to `https://github.com`
2. Click **Sign up**
3. Use the same email as Step 3
4. Complete verification

Step 5: Set Up SSH Authentication

Environment: Any terminal

```
$ ssh-keygen -t ed25519 -C "youremail@example.com"
```

Press Enter three times to accept defaults and skip passphrase.

Copy the public key:

Mac:

```
$ pbcopy < ~/.ssh/id_ed25519.pub
```

Windows (Git Bash):

```
$ cat ~/.ssh/id_ed25519.pub | clip
```

Linux:

```
$ cat ~/.ssh/id_ed25519.pub
```

(Select and copy the output manually)

Add to GitHub:

1. Go to `https://github.com` → **Settings**
2. **SSH and GPG keys** → **New SSH key**
3. Title: "My laptop"
4. Paste the key
5. Click **Add SSH key**

Test:

```
$ ssh -T git@github.com
```

Type `yes` , press Enter.

Expected Output:

```
Hi yourusername! You've successfully authenticated...
```

Step 6: Install Text Editor

Install VS Code from <https://code.visualstudio.com> . For this course, any text editor is acceptable.

Session 1: What is Git? Your First Repository

Learning Objectives

- Define version control using an analogy
- Initialize a Git repository
- Understand the staging area model
- Create commits with meaningful messages

Lecture: Theory (20 minutes)

Slide 1: The Chaos of No Version Control

Have you ever named a file:

```
report_final.doc  
report_final_v2.doc  
report_final_v2_ACTUAL.doc  
report_final_v2_ACTUAL_FOR_REAL.doc
```

This is **manual version control**. It works until you have:

- 50 files

- 3 people editing simultaneously
- A deadline in 2 hours

Slide 2: The Analogy — The Magic Camera

Analogy: *Git is like a magic camera. You work on your drawing, and when you reach a good stopping point, you take a photo. If you mess up later, you can say "Take me back to Photo #3!" and everything is restored.*

In Git:

- The **drawing** = your project files
- The **photo** = a commit
- The **photo album** = the repository
- The **camera itself** = the `.git` folder

Slide 3: The Staging Area Model

Git does not take photos automatically. You must:

1. **Create/edit files** (Working Directory — your messy desk)
2. **Put files in the box** (Staging Area — your prep zone)
3. **Take the photo** (Commit — permanent snapshot)

This three-stage model is the most important concept in Git.

Slide 4: Why "Staging"?

Imagine taking a family photo. You want everyone looking good, right? You don't just snap the camera when people are blinking. You:

1. Tell everyone to get ready (add to staging)
2. Check that everyone is smiling (`git status`)
3. Actually take the photo (`git commit`)

Slide 5: Commits Need Messages

Every photo needs a label:

- ☐ Bad label: "stuff", "changes", "asdf"
- ☐ Good label: "Add contact form", "Fix navigation CSS"

Think of it as writing on the back of the photo: "This was taken at Grandma's birthday."

Guided Lab: Your First Repository (25 minutes)

Environment: Terminal

Create a project folder:

```
$ cd ~  
$ mkdir devops-course  
$ cd devops-course
```

Check Git status before initialization:

```
$ git status
```

Expected Output:

```
fatal: not a git repository...
```

Initialize Git:

```
$ git init
```

Expected Output:

```
Initialized empty Git repository in .../.git/
```

Check status again:

```
$ git status
```

Expected Output:

```
On branch main  
No commits yet  
nothing to commit
```

Create your first file:

```
$ echo "Welcome to DevOps!" > readme.txt  
$ cat readme.txt
```

Check Git status:

```
$ git status
```

Expected Output:

```
Untracked files:  
  readme.txt
```

Stage the file:

```
$ git add readme.txt
```

Check status:

```
$ git status
```

Expected Output:

```
Changes to be committed:  
  new file:   readme.txt
```

Commit:

```
$ git commit -m "Add welcome message"
```

Expected Output:

```
[main (root-commit) abc1234] Add welcome message  
1 file changed, 1 insertion(+)
```

View history:


```
$ git log
```

Expected Output:

```
commit abc1234...
Author: Your Name <...>
Date: ...

    Add welcome message
```

Make another change:

```
$ echo "This course covers Git and GitHub." >> readme.txt
$ git status
```

Expected Output:

```
modified:   readme.txt
```

Stage and commit:

```
$ git add readme.txt
$ git commit -m "Add course description"
```

View compact log:

```
$ git log --oneline
```

Expected Output:

```
def5678 Add course description
abc1234 Add welcome message
```

Independent Exercise (10 minutes)

Create `tools.txt` with the text `Git, GitHub, CI/CD` . Stage and commit it.

▮ **Solution**

```
$ echo "Git, GitHub, CI/CD" > tools.txt
$ git add tools.txt
$ git commit -m "Add tools list"
```

Verify:

```
$ git log --oneline
```

****Expected Output:****

```
1234abc Add tools list
def5678 Add course description
abc1234 Add welcome message
```

Review (5 minutes)

Key questions:

1. What does `git init` do?
 2. What is the staging area?
 3. What is the difference between `git add` and `git commit` ?
 4. Why should commit messages be descriptive?
-
-

Session 2: Branching and Merging

Learning Objectives

- Explain branching with a "parallel universe" analogy
- Create and switch branches
- Merge branches back to main
- Understand why branching matters in team environments

Lecture: Theory (20 minutes)

Slide 1: The Problem with Working on Main

If three developers edit the same file simultaneously:

- Developer A rewrites the header
- Developer B adds a footer
- Developer C deletes the sidebar

They all save to `main` at the same time. Result: **Chaos**.

Slide 2: The Parallel Universe Analogy

Analogy: A branch is a parallel universe. In Universe A, you're adding a contact form. In Universe B, your teammate is redesigning the homepage. Neither universe affects the other. When both are ready, you merge the universes together.

Slide 3: Branch Naming Conventions

Good Name	Why
<code>feature-contact-form</code>	Clear what it does
<code>fix-broken-link</code>	Clear what it fixes
<code>update-about-page</code>	Clear what it updates

Bad Name	Why
<code>branch1</code>	Meaningless
<code>temp</code>	Will you remember what this was in 2 weeks?
<code>asdf</code>	Seriously?

Slide 4: Fast-Forward vs. Merge Commit

- **Fast-forward:** The main branch hasn't changed since you branched. Git just moves the pointer forward.

- **Merge commit:** The main branch has new commits. Git creates a new commit that combines both branches.

Guided Lab: Branching Workflow (25 minutes)

Environment: Terminal (in `~/devops-course`)

Verify you're on main:

```
$ git branch
```

Expected Output:

```
* main
```

Create a feature branch:

```
$ git branch feature-about  
$ git branch
```

Expected Output:

```
feature-about  
* main
```

Note the `*` is still on `main` . Switch:

```
$ git switch feature-about
```

Expected Output:

```
Switched to branch 'feature-about'
```

Verify:

```
$ git branch
```

Expected Output:

```
* feature-about  
main
```

Create a new file on the branch:

```
$ echo "About this course: learn Git from scratch." > about.txt  
$ git add about.txt  
$ git commit -m "Add about page"
```

Verify commit is on this branch:

```
$ git log --oneline
```

Check that main is unchanged:

```
$ git switch main  
$ ls
```

Expected Output: (no `about.txt`)

```
readme.txt  tools.txt
```

Switch back and merge:

```
$ git switch feature-about  
$ git switch main  
$ git merge feature-about
```

Expected Output:

```
Fast-forward  
about.txt | 1 +
```

Verify main now has the file:

```
$ ls
```

Expected Output:

```
about.txt  readme.txt  tools.txt
```

Delete the branch:

```
$ git branch -d feature-about
```

Check log:

```
$ git log --oneline
```

Independent Exercise (10 minutes)

Create a branch `feature-contact`. Add `contact.txt` with `Email: team@devops.com`. Commit, merge to `main`, delete the branch.

Solution

```
$ git branch feature-contact
$ git switch feature-contact
$ echo "Email: team@devops.com" > contact.txt
$ git add contact.txt
$ git commit -m "Add contact page"
$ git switch main
$ git merge feature-contact
$ git branch -d feature-contact
```

Verify:

```
$ ls
$ git log --oneline
```

Review (5 minutes)

1. Why do we use branches instead of editing `main` directly?
2. What does `git switch` do?
3. What is a fast-forward merge?
4. Why delete branches after merging?

Session 3: GitHub, Remotes, and Pull Requests

Learning Objectives

- Explain what a remote repository is
- Connect local repo to GitHub
- Push and pull code
- Create and merge Pull Requests

Lecture: Theory (20 minutes)

Slide 1: Local vs. Remote

- **Local repository:** Lives on your computer (your bedroom desk)
- **Remote repository:** Lives on GitHub (the school library)

In DevOps, the remote is the "source of truth" — the official version everyone trusts.

Slide 2: Why GitHub?

GitHub adds to Git:

- A web interface for viewing code
- Pull Requests (code review)
- Issues (bug tracking)
- Actions (automation/CI)
- Team collaboration tools

Slide 3: The Workflow

```
Local:    [create branch] → [make changes] → [commit]
                                     ↓
Remote:           [push]
                                     ↓
GitHub:  [open Pull Request] → [review] → [merge]
                                     ↓
Local:           [pull]
```

Slide 4: Pull Requests Explained

Analogy: A Pull Request is like a permission slip. You say: "I want to add my homework to the class folder. Please check it first." The teacher reviews it, and if it's correct, signs it (approves). Then it gets filed (merged).

In real DevOps, NO ONE pushes directly to `main`. All changes go through PR + code review.

Guided Lab: GitHub Workflow (25 minutes)

Environment: Browser + Terminal

Part A: Create GitHub Repository

1. Go to `https://github.com`
2. Click + → **New repository**
3. Name: `devops-course`
4. Description: "DevOps course project"
5. Public
6. Uncheck "Add a README file"
7. Click **Create repository**

Part B: Push Local Code

Environment: Terminal (in `~/devops-course`)


```
$ git remote add origin git@github.com:YOURUSERNAME/devops-course.git
$ git remote -v
```

Expected Output:

```
origin  git@github.com:../devops-course.git (fetch)
origin  git@github.com:../devops-course.git (push)
```

```
$ git push -u origin main
```

Expected Output:

```
* [new branch]      main -> main
```

Refresh GitHub. Your files are there!

Part C: Branch, Push, and PR

Create and switch to a branch:

```
$ git branch feature-faq
$ git switch feature-faq
```

Create and commit:

```
$ echo "Q: What is DevOps?" > faq.txt
$ echo "A: It's the practice of combining dev and ops." >> faq.txt
$ git add faq.txt
$ git commit -m "Add FAQ page"
```

Push branch:

```
$ git push -u origin feature-faq
```

Environment: Browser

1. On GitHub, click **"Compare & pull request"**
2. Title: Add FAQ page
3. Description:

- Added FAQ file - No breaking changes

4. Click **Create pull request**
5. Click **Files changed** to review
6. Click **Review changes** → **Approve** → **Submit review**
7. Click **Merge pull request** → **Confirm merge**
8. Click **Delete branch**

Part D: Pull Changes Locally

Environment: Terminal

```
$ git switch main  
$ git pull
```

Expected Output:

```
From github.com:.../devops-course  
... -> main  
Fast-forward  
faq.txt | 2 ++
```

Verify:

```
$ ls
```

Delete local branch:

```
$ git branch -d feature-faq
```

Independent Exercise (10 minutes)

Create a PR that adds `services.txt` with three services listed. Follow the full PR workflow.

□ Solution

```
$ git branch feature-services
$ git switch feature-services
$ cat > services.txt << 'EOF'
- Web Hosting
- CI/CD Pipelines
- Cloud Consulting
EOF
$ git add services.txt
$ git commit -m "Add services page"
$ git push -u origin feature-services
```

****On GitHub:**** - Create PR, review, merge, delete branch ****Back in terminal:****

```
$ git switch main
$ git pull
$ git branch -d feature-services
$ ls
```

Review (5 minutes)

1. What is a remote?
 2. What does `git push` do?
 3. What does `git pull` do?
 4. Why do we use Pull Requests instead of pushing directly?
-
-

Session 4: Undoing Changes and Handling Conflicts

Learning Objectives

- View commit history efficiently
- Revert commits safely
- Resolve merge conflicts
- Use git stash for temporary storage

Lecture: Theory (20 minutes)

Slide 1: The Philosophy of Undoing

Git gives you many ways to go back. Choosing the right one matters:

Situation	Right Tool	Why
Bad edit, not committed	<code>git restore</code>	Cleans working directory
Wrong file staged	<code>git restore --staged</code>	Removes from staging
Bad commit, local only	<code>git reset --soft HEAD~1</code>	Removes last commit
Bad commit, already pushed	<code>git revert</code>	Creates undo commit safely

Rule: *Never rewrite history that others have seen.*

Slide 2: Revert vs. Reset

- **Reset:** Erases commits (dangerous on shared history)
- **Revert:** Adds a new commit that undoes an old one (always safe)

In professional DevOps, you almost always use **revert**.

Slide 3: What is a Merge Conflict?

When two branches change the same line, Git says: "I don't know which version you want. You must tell me manually."

Analogy: Two people edit the same sentence in a shared document. The document can't decide which spelling to keep. A human must choose.

Slide 4: Git Stash

When you need to switch contexts quickly but aren't ready to commit:

1. **Stash** your current work (put it in a backpack)

2. Do the urgent thing
3. **Pop** the stash (unpack your backpack)

Guided Lab Part 1: Reverting (12 minutes)

Check current history:

```
$ cd ~/devops-course  
$ git log --oneline
```

Create a bad commit:

```
$ echo "WRONG CONTENT" > readme.txt  
$ git add readme.txt  
$ git commit -m "Accidentally overwrite readme"
```

Find the bad commit ID:

```
$ git log --oneline
```

Example Output:

```
bad9999 Accidentally overwrite readme  
... (older commits)
```

Revert it:

```
$ git revert bad9999
```

An editor opens. Save and exit:

- **Nano:** Ctrl+O, Enter, Ctrl+X
- **Vim:** Esc, `:wq`, Enter

Expected Output:

```
[main fix1234] Revert "Accidentally overwrite readme"
```

Verify the file:

```
$ cat readme.txt
```

Expected: Original content is restored.

Check log:

```
$ git log --oneline
```

Notice the bad commit is still there, plus the revert commit.

Push to GitHub:

```
$ git push
```

Guided Lab Part 2: Merge Conflict (13 minutes)

Create a scenario where conflict happens:

```
$ git branch conflict-demo  
$ git switch conflict-demo
```

Change a line:

```
$ echo "Branch version of readme" > readme.txt  
$ git add readme.txt  
$ git commit -m "Update readme on branch"
```

Switch to main and make a different change:

```
$ git switch main  
$ echo "Main version of readme" > readme.txt  
$ git add readme.txt  
$ git commit -m "Update readme on main"
```

Try to merge:

```
$ git merge conflict-demo
```

Expected Output:

```
CONFLICT (content): Merge conflict in readme.txt
Automatic merge failed
```

View the conflict:

```
$ cat readme.txt
```

Expected Output:

```
<<<<<<< HEAD
Main version of readme
=====
Branch version of readme
>>>>>>> conflict-demo
```

Resolve it by editing:

```
$ echo "Combined version of readme" > readme.txt
```

Mark resolved and commit:

```
$ git add readme.txt
$ git commit -m "Resolve readme conflict"
```

Independent Exercise (10 minutes)

Introduce a conflict on `about.txt` between `main` and a test branch. Resolve it by keeping the best parts of both versions.

□ Solution

```
$ git branch test-conflict
$ git switch test-conflict
$ echo "About: This is the test branch version." > about.txt
$ git add about.txt
$ git commit -m "Update about on test branch"

$ git switch main
$ echo "About: This is the main version." > about.txt
$ git add about.txt
$ git commit -m "Update about on main"

$ git merge test-conflict
# (conflict!)

$ echo "About: This course teaches Git. Test branch + main combined." > about.txt
$ git add about.txt
$ git commit -m "Resolve about conflict"
```

Review (5 minutes)

1. Why is `git revert` safer than `git reset`?
 2. What do conflict markers look like?
 3. After fixing a conflict, what two commands must you run?
 4. When would you use `git stash`?
-
-

Session 5: DevOps Patterns and GitHub Actions (CI/CD)

Learning Objectives

- Explain CI/CD with an analogy
- Write a GitHub Actions workflow in YAML
- Understand triggers and runners
- Set up basic branch protection

Lecture: Theory (20 minutes)

Slide 1: What is CI/CD?

- **CI (Continuous Integration):** Automatically test code on every change
- **CD (Continuous Deployment):** Automatically deploy if tests pass

Analogy: A robot watches the mailbox. Every time a letter arrives, it:

1. Opens the letter
2. Checks spelling
3. Makes a copy for records
4. Delivers it if everything looks good

Slide 2: Why DevOps Needs CI/CD

Without CI/CD	With CI/CD
Developer pushes code	Developer pushes code
Someone manually tests	Robot automatically tests
Days later, bug found	Minutes later, pass/fail known
Manual deployment	Automatic deployment

Slide 3: GitHub Actions Basics

- **Workflow:** A YAML file in `.github/workflows/`
- **Trigger:** Events like `push`, `pull_request`, `schedule`
- **Job:** A group of steps
- **Step:** A single command or action
- **Runner:** A VM that executes your workflow

Slide 4: Branch Protection

Branch protection rules:

- Require PR before merging
- Require status checks (CI must pass)
- Require code review

This prevents direct pushes to `main` and enforces quality.

Guided Lab: Build a CI Pipeline (25 minutes)

Environment: Terminal

Create workflow directory:

```
$ cd ~/devops-course  
$ mkdir -p .github/workflows
```

Write a workflow:

```
$ cat > .github/workflows/ci.yml << 'EOF'  
name: CI Check  
  
on:  
  push:  
    branches: [ main ]  
  pull_request:  
    branches: [ main ]  
  
jobs:  
  verify:  
    runs-on: ubuntu-latest  
    steps:  
      - uses: actions/checkout@v4  
      - name: Check files exist  
        run: |  
          test -f readme.txt && echo "readme.txt OK"  
          test -f about.txt && echo "about.txt OK"  
          test -f contact.txt && echo "contact.txt OK"  
          echo "All checks passed!"  
  
EOF
```

Commit and push:

```
$ git add .github/workflows/ci.yml  
$ git commit -m "Add CI workflow"  
$ git push
```

Environment: Browser

1. Go to **Actions** tab
2. Watch the workflow run
3. Click the run to see green checkmarks

Test Failure Scenario

Create a branch:

```
$ git branch break-build
$ git switch break-build
$ rm contact.txt
$ git add contact.txt
$ git commit -m "Remove contact (should fail CI)"
$ git push -u origin break-build
```

On GitHub:

- Open the PR
- CI will fail with red X
- Show the failed step
- Close the PR without merging

Set Branch Protection

1. Repo → **Settings** → **Branches**
2. Add rule: `main`
3. Check:
 - Require pull request
 - Require status checks: `CI Check`
4. Click **Create**

Verify by trying direct push:

```
$ git switch main
$ echo "test" >> readme.txt
$ git add readme.txt
$ git commit -m "Test direct push"
$ git push
```

Expected: Error — protected branch.

Undo the test commit:

```
$ git reset --soft HEAD~1
$ git restore --staged readme.txt
$ git checkout -- readme.txt
```

Independent Exercise (10 minutes)

Add a new status check step for `faq.txt` . Push and verify it runs on the Actions tab.

Solution

Edit `.github/workflows/ci.yml` to add:

```
- name: Check FAQ
  run: test -f faq.txt && echo "faq.txt OK"
```

```
$ git add .github/workflows/ci.yml
$ git commit -m "Add FAQ check to CI"
$ git push
```

Watch it on the Actions tab on GitHub.

Review and Capstone (5 minutes)

Students demonstrate:

1. All files in GitHub repo
2. At least one merged PR
3. Passing CI workflow
4. Branch protection enabled

Appendix A: Command Quick Reference

Command	Description
git init	Initialize a repository
git status	Check current state
git add <file>	Stage a file
git commit -m "msg"	Commit staged files
git log --oneline	Compact history
git branch <name>	Create branch
git switch <name>	Change branch
git merge <name>	Merge branch into current
git remote add origin <url>	Add remote
git push -u origin <branch>	Push branch to remote
git pull	Fetch and merge from remote
git revert <commit>	Undo a commit safely
git stash	Save temporary work
git stash pop	Restore stashed work

Appendix B: Session Timing

Session	Lect	Lab	Exercise	Review	Total
1	20	25	10	5	60
2	20	25	10	5	60
3	20	25	10	5	60
4	20	25	10	5	60
5	20	25	10	5	60

End of Approach 1: Linear Lecture + Lab